

INFO-F205 — Calcul Formel et Numérique

Syllabus d'exercices

Exercices initiaux de Maarten Jansen et Gianluca Bontempi
Corrections initiales par Keno Merckx et Arnaud Pollaris
Adaptation des énoncés et corrections et mise en page par Robin Petit

Année académique 2024-2025
Version du 9 février 2026

Table des matières

1	Introduction à Matlab/Octave	2
2	Représentation à virgule fixe et virgule flottante	6
2.1	Virgule fixe	7
2.2	Virgule flottante	7
3	Conditionnement et arithmétique tronquée	8
3.1	Conditionnement	11
3.2	Erreur d'annulation	11
3.3	Erreur de génération	12
4	Manipulation de nombres à virgule flottante en Matlab	13
5	Introduction à la résolution de systèmes linéaires	14
6	Résolution de systèmes linéaires par factorisation	17
7	Équations différentielles ordinaires	21
8	Interpolation	24
8.1	Interpolation de Lagrange	27
8.2	Interpolation de Newton	27
9	Interpolation (partie 2)	29
10	Résolution d'équations non-linéaires	33
11	Résolution d'équations non-linéaires par itération de point fixe	35

Séance 1 — Introduction à Matlab/Octave

Rappel théorique

Pour MATLAB (dont le nom vient de *MATrix LABoratory*), l'élément central pour les opérations est la matrice (ce qui comporte les vecteurs comme cas particuliers). La déclaration d'un vecteur se fait via les crochets :

```
x = [x1, x2, ...]
```

Les virgules pour séparer les éléments sont facultatives. L'instruction suivante est donc équivalente à la précédente :

```
x = [x1 x2 ...]
```

Une matrice se déclare comme une suite de vecteurs séparés par un point virgule. La matrice identité 2×2 peut donc se définir comme ceci :

```
I2 = [1 0; 0 1]
```

La notation `beg:end` (ou de manière plus générale `beg:step:end`) génère un vecteur contenant toutes les valeurs de la forme `beg + t*step` pour `t` entre 0 et $\lfloor (\text{end}-\text{beg})/\text{step} \rfloor$.

MATLAB propose toute une série de fonctions préexistantes (e.g. `sin`, `cos`, `exp`, `log`, `abs`, ...) qui peuvent être appelées tant sur des nombres que sur des matrices (et donc des vecteurs) en quel cas l'opération est effectuée individuellement sur chaque entrée de la matrice. Notez que `log(M)` ne calcule pas le log matriciel, il faut utiliser la fonction `logm` pour cela.

Il existe également des fonctions servant à générer des matrices dont `eye` qui génère la matrice identité (ou tout du moins une sous-matrice d'une matrice identité), `zeros` et `ones` qui génèrent respectivement une matrice ne contenant que des 0 ou que des 1, etc. Si un unique paramètre est donné à ces fonctions, alors elles renverront une matrice carrée dont la dimension correspond à ce paramètre. Si par contre plusieurs paramètres entiers sont donnés, alors la *matrice* (ou plus généralement le tableau de matrices) renvoyée aura les dimensions demandées.

Les dimensions d'une matrice peuvent être récupérées via la fonction `size` qui renvoie un vecteur dont les composantes correspondent aux dimensions. La fonction `length` quant à elle renvoie la plus grande des dimensions. La transposée d'une matrice (ou d'un vecteur) se fait via l'apostrophe.

```
>> A = zeros(2, 3)
A =
    0    0    0
    0    0    0
>> size(A)
ans =
     2     3
>> length(A)
```

```
ans = 3
>> A'
ans =
    0    0
    0    0
    0    0
```

Puisque MATLAB est prévu pour manipuler des matrices (et des vecteurs), les opérations usuelles sont celles définies sur les matrices. En particulier, si A et B sont deux matrices carrées, l'instruction $C = A*B$ va prendre le *produit matriciel* entre A et B et si c'est le produit de Hadamard qui est voulu (donc le *produit composante par composante*), il faut utiliser l'instruction $C = A .* B$. Nous pouvons mettre cela en lumière en ayant A la matrice identité $n \times n$ et B la matrice $n \times n$ ne contenant que des 1. La première est l'identité pour le produit matriciel alors que la seconde est l'identité pour le produit de Hadamard. Dès lors $A*B$ devrait donner B alors que $A .* B$ devrait donner A .

```
>> A = eye(2);
>> B = ones(2);
>> isequal(A * B, B)
ans = 1
>> isequal(A .* B, A)
ans = 1
```

De manière plus générale, précéder les opérateurs d'un point permet d'en utiliser la version *component-wise*. Remarquez qu'ici un point-virgule a été placé après les définitions de A et B . Cela permet de ne pas afficher le résultat de l'instruction. De plus, la fonction `isequal` est utilisée ici pour déterminer l'égalité entre plusieurs matrices (ou vecteurs). Cette fonction peut être appelée avec plus de deux paramètres et renvoie 1 si et seulement si toutes les matrices reçues ont les mêmes dimensions et les mêmes entrées aux mêmes indices.

Indicer une matrice (ou un vecteur) se fait avec des parenthèses (contrairement à la plupart des autres langages qui utilisent des crochets), et attention la première position correspond à l'indice 1 (et non 0)! Si $x = [1 \ 2 \ 3]$, alors $x(1)$ est 1 et $x(3)$ est 3. MATLAB permet un indigage un peu plus *fancy* : il est possible d'indicer une matrice avec un tableau afin de choisir les lignes (ou colonnes) à extraire. En particulier, puisque la notation `beg:end` (qui correspond à `beg:1:end`) crée un vecteur contenant toutes les valeurs entières entre `beg` et `end` avec un pas de 1, la notation $x(a:b)$ permet donc d'extraire un sous-vecteur de x en ne prenant que les indices entre a et b . Notons que le mot-clef `end` de MATLAB peut également servir lors de l'indigage : $x(2:end-1)$ permet par exemple de prendre tous les éléments du vecteur x sauf le premier et le dernier.

Exercice 1.1. Démarrez MATLAB (ou OCTAVE). Dans la fenêtre de commande, tapez $x = -1:0.1:1$ et ensuite, exécutez chacune des commandes suivantes :

```
sqrt(x)
sin(x)
x.^3
```

```
plot(x, cos(x .^ 2))
plot(x, sin(x .^ 3))
```

Exécutez les commandes suivantes et expliquez les résultats :

```
x = [2 3 4 5]
y = -1:1:2
x .^ y
x .* y
x ./ y
```

Exercice 1.2. Initialisez la matrice $A = [1 \ 5 \ 8; 84 \ 81 \ 7; 12 \ 34 \ 71]$ et examinez le contenu de :

- $A(1,1)$;
- $A(2,1)$;
- $A(1,2)$;
- $A(3,3)$;
- $A(1:2,:)$;
- $A(:,1)$;
- $A(3,:)$;
- $A(:,2:3)$;
- $A(1:end,:)$.

Que produisent les commandes suivantes :

```
x = 1:1:10
z = rand(10)
y = [z; x]
c = rand(4)
e = [c eye(size(c)); eye(size(c)) ones(size(c))]
d = sqrt(c)
t1 = d*d
t2 = d .* d
```

Exercice 1.3. Initialisez une matrice A de dimension 4×4 . Étant donné que la fonction `sum` donne la somme des éléments d'un vecteur donné en paramètre, utilisez la fonction `sum` pour trouver la somme des éléments de la première ligne et de la deuxième colonne. Attention à ne compter qu'une seule fois chaque élément !

Exercice 1.4. Soient les trois matrices suivantes :

$$A = \begin{bmatrix} 2 & -3 \\ 4 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 44 & 101 \\ -18 & -41 \end{bmatrix} \quad C = \begin{bmatrix} 1 & 2 \\ -2 & -5 \end{bmatrix}$$

À l'aide de MATLAB, calculez $C^{-1}AC$. Que pouvez-vous en déduire ?

Exercice 1.5. Écrivez un script qui prend en paramètre deux matrices carrées A et B et qui les additionne, les soustrait et les multiplie. Commentez votre script et utilisez la fonction `disp` pour l'affichage.

Exercice 1.6. Écrivez un script qui dessine les deux fonctions $f_1(x) = x^2 \cos x$ et $f_2(x) = x^2 \sin x$ sur la même figure. Prenez $x = -2:0.1:2$.

Exercice 1.7. L'équation itérative suivante permet de résoudre l'équation $x^2 - x - 1 = 0$:

$$x_{t+1} = 1 + \frac{1}{x_t}$$

pour $t \geq 0$, où $x_0 = 2$. Écrivez un script MATLAB qui implémente ceci à précision ε choisie. Testez votre programme pour $\varepsilon = 5 \times 10^{-4}$.

Exercice 1.8. Étant donné A , une matrice de dimension 4×5 , écrivez un script pour trouver la somme de chacune des colonnes en utilisant (i) une boucle `for` et (ii) la fonction `sum`.

Exercice 1.9. Soit x un vecteur de n éléments. Écrivez un script MATLAB pour déterminer le produit

$$P_k = \prod_{i \neq k} x_i = x_1 x_2 \dots x_{k-1} x_{k+1} \dots x_{n-1} x_n.$$

Testez votre script avec des valeurs extrêmes de x et de n .

Exercice 1.10. Écrivez un script qui génère une matrice dont les éléments le long de la diagonale valent d et les coefficients directement au-dessus et en-dessous de cette diagonale valent c . Le reste des entrées doit être égal à 0.

Exercice 1.11. Écrivez une fonction MATLAB pour résoudre l'équation suivante :

$$ax^2 + bx + c = 0.$$

La fonction doit prendre en entrée les paramètres a , b et c et renvoyer les valeurs des deux racines

Exercice 1.12. En sachant que la série suivante :

$$\sum_{k \geq 0} A^k = I + A + A^2 + A^3 + \dots$$

converge vers $(I - A)^{-1}$ si les valeurs propres de la matrice A sont bornées par 1 en valeur absolue. La matrice suivante (de dimension $n \times n$) satisfait cette condition si $a + 2b < 1$:

$$\begin{bmatrix} a & b & 0 & \dots & 0 & 0 & 0 \\ b & a & b & \dots & 0 & 0 & 0 \\ 0 & b & a & \dots & 0 & 0 & 0 \\ \vdots & & & \ddots & & & \vdots \\ 0 & 0 & 0 & \dots & a & b & 0 \\ 0 & 0 & 0 & \dots & b & a & b \\ 0 & 0 & 0 & \dots & 0 & b & a \end{bmatrix}$$

Expérimentez avec cette matrice pour différentes valeurs de a , b et n pour illustrer que cette série converge sous la condition annoncée.

Séance 2 — Représentation à virgule fixe et virgule flottante

Rappel théorique

Dans ce qui suit, nous devons toujours travailler dans une base b fixée (typiquement $b = 2$ dans un ordinateur ou $b = 10$ lorsque nous raisonnons sur papier).

La représentation à *virgule fixe* consiste à se fixer une base $b \geq 2$, un signe $s \in \{0, 1\}$ (où $s = 0$ désigne qu'un nombre est positif et $s = 1$ désigne qu'un nombre est négatif) et $n + m + 1$ chiffres (en base b toujours) dont les $n + 1$ premiers sont utilisés pour représenter la partie entière d'un nombre et les m suivants sont utilisés pour représenter la partie décimale, i.e. :

$$x = ([x_n, x_{n-1}, \dots, x_1, x_0, x_{-1}, \dots, x_{-m}], b, s),$$

où les x_k sont entre 0 et $b - 1$ (donc des chiffres en base b). Cette représentation représente le nombre :

$$x = (-1)^s \sum_{k=-m}^n x_k \cdot b^k.$$

Attention, nous identifions ici la représentation et la valeur du nombre représenté.

Travailler en virgule fixe correspond en réalité à travailler uniquement sur des entiers sur $m + 1 + n$ chiffres, mais de les interpréter $\times b^{-m}$.

Cependant, en fonction du contexte, on pourrait vouloir avoir plus de précision pour la partie entière ou plus de précision pour la partie décimale, ce qui n'est pas possible.

La *virgule flottante* cherche à contourner ce problème en travaillant sur une *mantisse* de taille t fixée, mais également avec un exposant e (généralisant la notation scientifique). Le nombre à virgule flottante suivant

$$x = ([x_1, \dots, x_t], e, b, s)$$

est donc à interpréter comme le nombre réel suivant

$$x = (-1)^s m b^{e-t} = (-1)^s \sum_{k=1}^t x_k b^{e-k}.$$

Attention, nous imposons à nouveau ici $s \in \{0, 1\}$, $b \geq 2$, $e \in \mathbb{Z}$ (en effet cet exposant peut très bien être négatif), mais également $1 \leq x_1 \leq b - 1$ et $0 \leq x_k \leq b - 1$ pour $k \geq 2$. En particulier x_1 ne peut pas être nul (ce qui garantit l'unicité de la représentation d'un nombre sous cette forme).

À base b fixée, pour t, L, U des entiers tels que $L \leq U$, nous définissons l'ensemble $\mathbb{F}(b, t, L, U)$ comme l'ensemble contenant tous les réels $x \in \mathbb{R}$ qui admettent une représentation à virgule flottante avec (i) une mantisse de taille t ; (ii) la base b ; et (iii) $L \leq e \leq U$.

Notons qu'il est possible de montrer que $|\mathbb{F}(b, t, L, U)| = 2(U - L + 1)(b - 1)b^{t-1}$. En effet, dans la représentation $([x_1, \dots, x_t], e, b, s)$ peut prendre :

- 2 valeurs pour s (à savoir 0, 1) ;

- $U - L + 1$ valeurs pour e (à savoir $L, L + 1, \dots, U - 1, U$);
- $b - 1$ valeurs pour x_1 (à savoir $1, 2, \dots, b - 1$);
- et b valeurs pour chacun des x_k pour $2 \leq k \leq t$ (à savoir $0, 1, \dots, b - 1$).

Le produit de ces différentes entrées vaut bien $2(U - L + 1)(b - 1)b^{t-1}$.

Notons tout de même que le fait d'imposer $x_1 \neq 0$ nous empêche de représenter 0 en virgule flottante. Nous appelons ces représentations sous *forme normalisée*. Évidemment, en pratique, la possibilité de tout de même mettre x_1 à 0 permet de représenter des *valeurs spéciales*, e.g. la valeur 0, mais également $\pm\infty$, ou encore NaN.

2.1 Virgule fixe

- **Exercice 2.1.** Si $b = 10, n = 3, m = 6$ et $s = 0$, à quel nombre correspond $[0012200000]$?
- **Exercice 2.2.** Même question si $b = 3$?
- **Exercice 2.3.** Si $b = 3, n = 3, m = 6$ et $s = 0$, à quel nombre correspond $[3012200000]$?

2.2 Virgule flottante

■ **Exercice 2.4.** A quels nombres correspondent :

- $([34], e = 2, b = 10, s = 0)$;
- $([34], e = 1, b = 16, s = 1)$;
- $([10001], e = 6, b = 2, s = 0)$?

■ **Exercice 2.5.** Représentez le nombre 234.5 en virgule flottante avec $b = 2$ et les autres paramètres suffisamment grands.

■ **Exercice 2.6.** Combien de nombres peut-on représenter en virgule flottante en base $b = 4$, et avec les paramètres $L = -2, U = 2$, et $t = 2$?

■ **Exercice 2.7.** Considérons le système linéaire suivant :

$$\begin{cases} 0.461x_1 + 0.311x_2 &= 0.150 \\ 0.209x_1 + 0.141x_2 &= 0.068 \end{cases}$$

■ Il admet pour solution $(x_1, x_2) = (1, -1)$. Montrez que la résolution de ce système en utilisant une représentation en virgule flottante ($b = 10, t = 3$) manque de précision.

■ **Exercice 2.8.** Considérez l'équation $\alpha x + \beta = 0$ pour $\alpha = 0.111$ et $\beta = -2$. $z = 18$ en est une solution approchée. Utilisez l'arithmétique décimale (donc en base $b = 10$) tronquée à 3 chiffres (donc $t = 3$) pour calculer le reste $r = -\beta - \alpha \cdot z$. Comparez avec le résultat en arithmétique exacte.

Séance 3 — Conditionnement et arithmétique tronquée

Rappel théorique

Conditionnement

Nous considérons un *problème* au sens mathématique, c'est-à-dire une expression sous la forme

$$F(x, d) = 0$$

pour $F : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$ une fonction et $d \in \mathbb{R}^m$ le vecteur représentant les *données*. Résoudre ce problème correspond à trouver une valeur $x \in \mathbb{R}^n$ satisfaisant cette équation.

Si nous pouvons réécrire cette expression sous la forme

$$x = F(d),$$

nous disons que le problème est en *forme explicite*, et sinon il est en *forme implicite*. Nous considérons généralement des problèmes en forme explicite.

Nous notons δd une *perturbation* des données d , et nous notons δx la *perturbation* associée de la solution. Plus précisément, si $\hat{d} = d + \delta d$ est la donnée sur laquelle nous travaillons, alors nous définissons

$$\delta x = F(\hat{d}) - x = F(\hat{d}) - F(d) = F(d + \delta d) - F(d).$$

Nous écrivons également cela

$$x + \delta x = F(d + \delta d).$$

Si $\|x\| \neq 0$ et $\|d\| \neq 0$, nous définissons le *conditionnement relatif* du problème F en la donnée d par :

$$\begin{aligned} \kappa_{\text{rel}}(d) &:= \lim_{D \rightarrow 0} \sup_{\delta d \in B(0, D)} \frac{\|\delta x\| / \|x\|}{\|\delta d\| / \|d\|} \\ &= \frac{\|d\|}{\|x\|} \lim_{D \rightarrow 0} \sup_{\delta d \in B(0, D)} \frac{\|\delta x\|}{\|\delta d\|}. \end{aligned}$$

En particulier si $F \in \mathcal{C}^1(\mathbb{R}, \mathbb{R})$, alors :

$$\kappa_{\text{rel}}(d) = \frac{|d|}{|x|} \lim_{\delta d \rightarrow 0} \left| \frac{F(d + \delta d) - F(d)}{\delta d} \right| = \left| \frac{d \cdot F'(d)}{F(d)} \right|.$$

S'il n'est pas possible de regarder le conditionnement relatif, nous pouvons également regarder le *conditionnement absolu* :

$$\kappa_{\text{abs}}(d) := \lim_{D \rightarrow 0} \sup_{\delta d \in B(0, D)} \frac{\|\delta x\|}{\|\delta d\|}.$$

Notez bien que la raison pour laquelle nous passons par $\lim_{D \rightarrow 0} \sup_{\delta d \in B(0,D)}(\dots)$ dans ces définitions est pour ne pas s'encombrer du cas où $\lim_{\delta d \rightarrow 0}(\dots)$ n'existe pas. En effet pour une suite $(D_n)_n$ décroissante, $n \mapsto \sup_{\delta d \in B(0,D_n)} \varphi(\delta d)$ est également une suite décroissante, et donc cette suite converge (potentiellement vers $+\infty$ si toutes les valeurs sont infinies).

Erreurs

À partir d'ici, fixons un certain $x \in \mathbb{R}$, et considérons $\hat{x} \approx x$ une approximation de x . Nous appelons

- *erreur absolue* la quantité $\delta = |\hat{x} - x|$;
- *écart relatif* la quantité $\rho = \frac{\hat{x} - x}{x}$;
- *erreur relative* la quantité $\varepsilon = |\rho| = \frac{|\hat{x} - x|}{|x|} = \frac{\delta}{|x|}$.

Dans le cas où nous parlons d'erreurs ou d'écarts sur plusieurs quantités différentes, nous ajoutons le nom de la variable considérée en indice, e.g. δ_x ou encore ρ_d .

Notons que $\rho_x = \frac{\hat{x} - x}{x}$, et donc $x\rho_x = \hat{x} - x$, ou encore $\hat{x} = x\rho_x + x = x(1 + \rho_x)$.

Si $x \in \mathbb{R} \setminus \mathbb{F}(b, t, L, U)$, nous ne pouvons représenter qu'une valeur approchée de x . Nous considérons ici les cas suivants :

- un *arrondi* : $\hat{x} = \text{fl}(x)$;
- une *valeur tronquée* : $\hat{x} = \text{tr}(x)$.

Dans le cas d'un arrondi, nous prenons la valeur $\hat{x} \in \mathbb{F}(b, t, L, U)$ qui minimise la distance $|\hat{x} - x|$, alors que dans le cas d'une troncature, nous prenons le plus grand nombre $\hat{x}$ tel que $\hat{x} \leq x$.

Si x et x' sont deux valeurs consécutives de $\mathbb{F}(b, t, L, U)$ (dans le sens où $x < x'$ et il n'existe aucun $\xi \in \mathbb{F}(b, t, L, U)$ tel que $x < \xi < x'$), alors nous pouvons montrer les inégalités suivantes :

$$\frac{1}{b} \varepsilon \leq \frac{|x - x'|}{|x|} \leq \varepsilon,$$

où la quantité $\varepsilon := b^{1-t}$ est appelée *epsilon machine*. De plus, dans le cas d'un arrondi, nous pouvons observer que :

$$\left| \frac{\text{fl}(x) - x}{x} \right| \leq u,$$

où la quantité $u := \frac{1}{2}\varepsilon$ est appelée *précision machine* (ou parfois *unit roundoff* d'où la lettre u).

Erreur de propagation Si nous travaillons sur la valeur $\hat{d} = \text{fl}(d)$ (au lieu de d) et que nous résolvons le problème $F(d)$ associé, nous obtenons une solution $x + \delta x$. L'*erreur de propagation* est l'erreur obtenue sur x , exprimé sur base de l'erreur sur d :

$$\begin{aligned} F(\hat{d}) &= F(d + \delta d) = F(d(1 + \rho_d)) = F(d + d \cdot \rho_d) \\ &\approx F(d) + d\rho_d F'(d) = F(d) + F(d) \frac{d\rho_d F'(d)}{F(d)} = F(d) \left(1 + \frac{d\rho_d F'(d)}{F(d)} \right) \\ &= x(1 + \rho_d \kappa(d)) \end{aligned}$$

où le symbole $\approx$ cache en réalité l'application d'une expansion de Taylor d'ordre 1.

Finalement, la solution *obtenue* est :

$$\hat{x} = \text{fl}(F(\hat{d})) = \text{fl}(x(1 + \rho_d \kappa(d))) = x(1 + \rho_d \kappa)(1 + \rho_{F(d)}).$$

Les erreurs d'*annulation* et d'*absorption* sont deux exemples importants d'erreurs de génération. Une erreur d'annulation arrive lorsque deux nombres proches sont soustraits. Dans ce cas, la valeur résultante est certes petite en valeur absolue, mais peut être potentiellement très grande par rapport à la *vraie* valeur de cette différence, surtout dans le cas où les opérandes sont déjà des valeurs arrondies. Une erreur d'absorption arrive lorsque qu'un nombre très grand est ajouté à un nombre très petit. Dans ce cas, par manque de chiffres significatifs, une grande partie des chiffres composant la *petite* valeur n'apparaîtront pas dans la valeur résultante.

Erreur de génération L'erreur de génération apparaît lorsque le problème $x = F(d)$ est résolu par un algorithme itératif :

$$\begin{aligned} x_1 &= F_1(d) \\ x_2 &= F_2(x_1) \\ &\vdots \\ x &= x_n = F_n(x_{n-1}) \end{aligned}$$

Ici, des erreurs sont produites à chaque étape, et nous pouvons les calculer comme ceci :

$$\hat{x}_1 = x_1(1 + \kappa_1 \rho_d)(1 + \rho_{x_1}) = x_1(1 + \kappa_1 \rho_d \rho_{x_1} + \kappa_1 \rho_d + \rho_{x_1}) \approx x_1(1 + \underbrace{\kappa_1 \rho_d + \rho_{x_1}}_{=:\rho_{x_1}^C}),$$

où nous prenons l'approximation $\rho_a \times \rho_b \approx 0$ (l'ordre de grandeur des erreurs relatives est suffisamment grand pour être pris en compte, mais suffisamment petit pour que le produit de deux telles erreurs puisse être approximé à 0) et où $\rho_{x_1}^C$ est l'écart relatif cumulé (total) sur x_1 . Dès lors :

$$\begin{aligned} \hat{x}_2 &\approx x_2(1 + \kappa_2 \rho_{x_1}^C)(1 + \rho_{x_2}) \\ &\vdots \\ \hat{x} &\approx x \left[1 + \underbrace{\underbrace{\rho_n}_{\text{arrondi}} + \sum_{k=1}^{n-1} \left(\prod_{m=k+1}^n \kappa_m \right) \rho_k + \rho_d \prod_{m=1}^n \kappa_m}_{=:\rho_{x_n}^C} \right]. \end{aligned}$$

Nous observons que le conditionnement des différents sous-problèmes finissent multipliés, et donc si ces sous-problèmes sont mal conditionnés, $|\hat{x} - x|$ peut se retrouver arbitrairement grand.

Remarque. Pour une preuve de cette assertion, nous pouvons procéder par récurrence sur n . Nous voulons montrer que :

$$\frac{\hat{x}_j}{x_j} - 1 \approx \rho_{x_j}^C = \rho_j + \sum_{k=1}^{j-1} \left(\prod_{m=k+1}^j \kappa_m \right) \rho_k + \rho_d \prod_{m=1}^j \kappa_m.$$

Comme vu ci-dessus, $\hat{x}_1 \approx x_1 (1 + \rho_1 + \kappa_1 \rho_d) = x_1 (1 + \rho_{x_1}^C)$.

$$\begin{aligned}
\hat{x}_j &= x_j \left(1 + \kappa_j \left(\frac{\hat{x}_{j-1}}{x_{j-1}} - 1 \right) \right) (1 + \rho_j) \\
&= x_j \left(1 + \kappa_j \rho_{x_{j-1}}^C \right) (1 + \rho_j) \\
&\approx x_j \left[1 + \rho_j + \kappa_j \left(\rho_{j-1} + \sum_{k=1}^{j-2} \left(\prod_{m=k+1}^{j-1} \kappa_m \right) \rho_k + \rho_d \prod_{m=1}^{j-1} \kappa_m \right) \right] \\
&= x_j \left[1 + \rho_j + \left(\prod_{m=j}^j \kappa_m \right) \rho_{j-1} + \sum_{k=1}^{j-1} \left(\prod_{m=k+1}^{j-1} \kappa_m \right) \kappa_j \rho_k + \rho_d \prod_{m=1}^j \kappa_j \right] \\
&= x_j \left[1 + \rho_j + \sum_{k=1}^{j-1} \left(\prod_{m=k+1}^j \kappa_m \right) \rho_k + \rho_d \prod_{m=1}^j \kappa_m \right] \\
&= x_1 \left(1 + \rho_{x_j}^C \right).
\end{aligned}$$

Notations Pour une opérations $\star : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$, nous considérons l'opération associée $\oplus : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{F}$ définie comme suit :

$$x \oplus y := \text{fl}(\text{fl}(x) \star \text{fl}(y)).$$

En particulier $\oplus$ désigne l'addition arrondie et $\odot$ désigne la multiplication arrondie. Maintenant par induction :

3.1 Conditionnement

Exercice 3.1. Que peut-on dire du conditionnement de la fonction $F(d) = d-1$ autour de 1 ?

Exercice 3.2. Calculez le conditionnement $\kappa(d)$ des expressions $F(x, d)$ suivantes :

- $x - a^d = 0$ ($a > 0$) ;
- $d - x + 1 = 0$.

Exercice 3.3. Montrez que l'évaluation de $F(d) = \cos d$ est bien conditionnée près de $d = 0$.

Exercice 3.4. Que peut-on dire du conditionnement de $F(d) = (d-1)^2$ autour de $d = 1$?

3.2 Erreur d'annulation

Exercice 3.5. Vérifiez que pour $d_1 = 0.1656$, $d_2 = 7.4409$ et $d_3 = 2.9168$ dans un système à virgule flottante où $b = 10$ et $t = 2$:

$$d_1 \oplus (d_2 \oplus d_3) \neq (d_1 \oplus d_2) \oplus d_3,$$

i.e. $\oplus$ n'est pas associatif.

Trouvez un exemple de valeurs distinctes x, y, z satisfaisant :

$$x \oplus (y \oplus z) = (x \oplus y) \oplus z.$$

Exercice 3.6. Pour un système en virgule flottante où $b = 10$ et $t = 4$ montrez que l'évaluation de la fonction $F(d) = d - 1$ est sujette à une importante erreur d'annulation dans le cas où $d = 1.00098$.

3.3 Erreur de génération

Exercice 3.7. Pour $\alpha = 0.8717$ et $\beta = 0.8719$, calculez le point milieu de l'intervalle $(\alpha + \beta)/2$. Utilisez d'abord l'arithmétique décimale *tronquée* à 4 chiffres, et ensuite l'arithmétique décimale *arrondie* à 4 chiffres. Enfin, proposez une autre séquence d'évaluation qui génère moins d'erreur.

Exercice 3.8. Dans un système en virgule flottante où $b = 10$ et $t = 3$, comparez les deux algorithmes suivants d'évaluation de la fonction $F(d) = d^2 - 1$:

- algorithme 1 : $y_1 = d \cdot d$; $x_1 = y_1 - 1$;
- algorithme 2 : $y_2 = d - 1$; $x_2 = y_2 \cdot (y_2 + 2)$.

Séance 4 — Manipulation de nombres à virgule flottante en Matlab

Dans cette séance, nous ferons l'hypothèse simplificatrice que l'arithmétique employée par MATLAB (ou OCTAVE) est exacte.

Exercice 4.1. Écrivez une fonction qui, étant donné un nombre x et son approximation $\hat{x}$, calcule (i) l'erreur absolue et (ii) l'erreur relative.

Exercice 4.2. Dessinez le graphique de de l'erreur relative obtenue pour le problème $F(d) = (d - 1)^2$ avec une perturbation fixe des données de 10^{-2} . Prenez les données dans l'intervalle $[-1, 3]$. Dessinez ensuite sur la même figure le graphique du conditionnement relatif du problème.

Exercice 4.3. Écrivez une fonction `increment(m, b)` qui prend en paramètre une mantisse m sous forme d'un vecteur de taille t et une base b , et qui renvoie la mantisse associée à $m+1$ (toujours en base b).

Ensuite, écrivez une fonction `floset(b, t, L, U)` qui prend en paramètre une base b , une taille de mantisse t , et les bornes sur un exposant L et U . Cette fonction doit renvoyer une paire $[F, \text{card}]$ où F est un vecteur trié qui contient toutes les valeurs de $\mathbb{F}(b, t, L, U)$ et où `card` est la taille de F .

Représentez ensuite ce vecteur sur une figure (c.f. la Figure 2.1 du syllabus).

Exercice 4.4. Utilisez le code de l'exercice précédent et représentez sur une figure les distances relatives entre les éléments consécutifs de $\mathbb{F}(b, t, L, U)$ (c.f. la Figure 2.2 du syllabus).

Exercice 4.5. Écrivez une fonction `fl(x, b, t, L, U)` qui prend en paramètre une valeur $x \in \mathbb{R}$, une base b , une taille de mantisse t et des bornes L et U sur un exposant ; et qui renvoie la valeur $\hat{x} = \text{fl}(x) \in \mathbb{F}(b, t, L, U)$, i.e. la valeur approchée la plus proche de x dans cette représentation.

Séance 5 — Introduction à la résolution de systèmes linéaires

Rappel théorique

Un système linéaire est un ensemble de m équations linéaires à n inconnues :

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m \end{cases}$$

Nous pouvons le réécrire sous forme matricielle :

$$Ax = b,$$

où $A \in \mathbb{R}^{m \times n}$ est la matrice de coefficients, $x \in \mathbb{R}^n$ est le vecteur d'inconnues et $b \in \mathbb{R}^m$ est le vecteur de solutions.

Théorème 1. Si A est une matrice carrée, alors les conditions suivantes sont équivalentes :

1. A est inversible (ou encore $\det A \neq 0$);
2. $Ax = 0$ admet uniquement $x = 0$ pour solution;
3. $Ax = b$ admet une unique solution pour tout b ;
4. $\text{rank } A = m = n$.

Méthode de Cramer

Nous introduisons la notation suivante pour tout $1 \leq j \leq n$:

$$B_j := \begin{bmatrix} a_{1,1} & \dots & a_{1,j-1} & b_1 & a_{1,j+1} & \dots & a_{1,n} \\ & & & \vdots & & & \\ a_{n,1} & \dots & a_{n,j-1} & b_n & a_{n,j+1} & \dots & a_{n,n} \end{bmatrix}$$

La matrice B_j est donc la matrice obtenue en remplaçant la j ème colonne de A par le vecteur b .

Théorème 2 (Cramer). Le vecteur x donné par $x_j = \frac{\det B_j}{\det A}$ est une solution à l'équation $Ax = b$.

Méthode de Gauss

Une matrice A est dite *triangulaire supérieure* si $a_{i,j} = 0$ pour tous $i > j$ et est dite *triangulaire inférieure* si $a_{i,j} = 0$ pour tous $j > i$. A est dite *triangulaire* si elle est soit triangulaire supérieure soit triangulaire inférieure.

Observons que si la matrice A est triangulaire, le système $Ax = b$ est facile à résoudre. En effet, si A est triangulaire supérieure, nous utilisons la substitution rétrograde :

$$x_i = \frac{1}{a_{i,i}} \left(b_i - \sum_{j=i+1}^n a_{i,j}x_j \right).$$

En déterminant les entrées de x dans l'ordre décroissante (de x_n jusque x_1), nous pouvons trouver la solution en temps quadratique. De manière symétrique, si A est triangulaire inférieure, nous utilisons la substitution progressive :

$$x_i = \frac{1}{a_{i,i}} \left(b_i - \sum_{j=1}^{i-1} a_{i,j} x_j \right).$$

En déterminant les entrées de x dans l'ordre croissant (de x_1 à x_n), à nouveau, nous pouvons trouver la solution en temps quadratique.

L'objectif de la méthode de Gauss est de transformer le système $Ax = b$ en un système équivalent $A'x = b'$ où la matrice A' est triangulaire. Ici, deux systèmes $A^{(1)}x = b^{(1)}$ et $A^{(2)}x = b^{(2)}$ (ou de manière équivalente deux paires $(A^{(1)}, b^{(1)})$ et $(A^{(2)}, b^{(2)})$) sont dits *équivalents* lorsqu'ils admettent les mêmes solutions.

À l'étape $j \in \{1, \dots, n\}$, on veut s'assurer que la matrice A soit triangulaire (supérieure) jusqu'à la colonne j , i.e. que pour $j' \leq j$ et $i > j'$, l'entrée $A_{i,j'}^{(j)}$ soit nulle. En procédant de la sorte, à la fin (donc après la n ème étape), nous aurons bien une matrice triangulaire supérieure.

Pour ce faire, nous notons $m_{i,j}$ le coefficient utilisé à la ligne i , lors de l'étape j , et nous définissons la matrice $A^{(j)}$ comme ceci (où $A^{(0)} := A$) :

$$a_{i,k}^{(j)} = \begin{cases} a_{i,k}^{(j-1)} - m_{i,j} a_{j,k}^{(j-1)} & \text{si } i > j \\ a_{i,k}^{(j-1)} & \text{si } i \leq j \end{cases}$$

De manière équivalente, nous définissons $A^{(j)}$ comme étant $A^{(j-1)}$ à laquelle la j ème ligne est retirée $m_{i,j}$ fois à la ligne i . Bien entendu la même opération doit être appliquée sur b et donc (pour $b^{(0)} := b$) :

$$b_i^{(j)} = \begin{cases} b_i^{(j-1)} - m_{i,j} b_j^{(j-1)} & \text{si } i > j \\ b_i^{(j-1)} & \text{si } i \leq j \end{cases}$$

Nous voulons nous assurer que $a_{i,k}^{(j)} = 0$ pour tous i, j, k tels que $i > k$ et $k \leq j$. Pour cela, nous prenons $m_{i,j} = \frac{a_{i,j}^{(j-1)}}{a_{j,j}^{(j-1)}}$. Nous pouvons aisément vérifier que c'est suffisant par induction. En effet pour $j = 0$, le résultat est trivialement vrai. Pour $j \geq 1$, soit nous regardons le cas $k = j$ où donc pour $i > k$:

$$a_{i,k}^{(j)} = a_{i,j}^{(j)} = a_{i,j}^{(j-1)} - m_{i,j} a_{j,j}^{(j-1)} = 0,$$

par définition de $m_{i,j}$, soit nous regardons le cas $k < j$ et donc par hypothèse d'induction, pour $i > k$:

$$a_{i,k}^{(j)} = \begin{cases} a_{i,k}^{(j-1)} = 0 & \text{si } i \leq j \\ a_{i,k}^{(j-1)} - m_{i,j} a_{j,k}^{(j-1)} = 0 - m_{i,j} \cdot 0 = 0 & \text{si } i > j \end{cases}$$

Attention tout de même, ici à chaque étape, nous devons diviser par $a_{j,j}^{(j-1)}$, mais donc si cette valeur est nulle, le problème est mal défini. Il nous faut alors utiliser la notion de *pivot*, i.e. nous devons interchanger les lignes j et j' où $j' > j$ est l'indice maximisant la

quantité $|a_{j',j}^{(j-1)}|$. Cette méthode de pivot s'appelle *pivot partiel*. Il existe également une notion de *pivot total* dans lequel on échange à la fois deux lignes et deux colonnes afin de placer l'entrée $a_{i',j'}^{(j-1)}$ maximale en valeur absolue à la place de $a_{j,j}^{(j-1)}$. Dans ce cas-ci, attention tout de même à bien conserver les permutations de colonnes qui ont été faites afin de pouvoir retrouver la bonne valeur de x (puisque un échange de colonnes correspond à intervertir deux entrées de x).

Exercice 5.1. Résolvez le système suivant avec la méthode de Cramer :

$$\begin{bmatrix} 4 & 8 & 12 \\ 3 & 8 & 13 \\ 2 & 9 & 18 \end{bmatrix} x = \begin{bmatrix} 4 \\ 5 \\ 11 \end{bmatrix}.$$

■ **Exercice 5.2.** Résolvez ce même système par la méthode de Gauss.

Exercice 5.3. Résolvez le système suivant avec la méthode de Gauss en arithmétique flottante pour $b = 10$ et $t = 3$:

$$\begin{bmatrix} 10^{-4} & 1 \\ 1 & 1 \end{bmatrix} x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Faites pareil en échangeant les deux lignes.

Exercice 5.4. Résolvez les systèmes suivantes avec la méthode de Gauss et pivot partiel :

1.

$$\begin{bmatrix} -1 & 2 & 1 \\ 1 & 4 & -3 \\ -2 & 0 & 1 \end{bmatrix} x = \begin{bmatrix} 5 \\ -8 \\ 5 \end{bmatrix}$$

2.

$$\begin{bmatrix} 1 & 2 & -1 \\ 2 & 4 & 1 \\ 3 & 6 & -2 \end{bmatrix} x = \begin{bmatrix} 2 \\ 7 \\ 7 \end{bmatrix}$$

3.

$$\begin{bmatrix} 1 & -1 & 1 \\ 2 & 1 & -1 \\ 1 & 2 & -2 \end{bmatrix} x = \begin{bmatrix} 0 \\ -3 \\ -2 \end{bmatrix}$$

4.

$$\begin{bmatrix} 2 & -3 & 2 & 5 \\ 1 & -1 & 1 & 2 \\ 3 & 2 & 2 & 1 \\ 1 & 1 & -3 & -1 \end{bmatrix} x = \begin{bmatrix} 3 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Séance 6 — Résolution de systèmes linéaires par factorisation

Rappel théorique

Factorisation LU

Observons que $\det(AB) = \det A \det B$ pour toutes matrices carrées A et B . Dès lors, si $\det A$ et $\det B$ sont *faciles* à calculer, le déterminant de AB devient également simple à déterminer. Dans l'autre sens, si en partant d'une matrice A , on peut la factoriser en le produit $A = LU$, telles que $\det L$ et $\det U$ sont faciles à calculer, alors $\det A$ devient également facile à calculer. En particulier, si L et U sont triangulaires, le déterminant de A est également facile à calculer puisque le déterminant d'une matrice triangulaire est simplement le produit des éléments sur sa diagonale.

Une autre justification pour l'intérêt de la factorisation LU est de réaliser que lors de l'élimination de Gauss faite au TP précédent, seule la matrice A nous intéresse, le vecteur b est modifié également, mais ses valeurs ne sont utilisées que lors de la résolution. Dès lors, s'il y a plusieurs systèmes linéaires à résoudre à la suite les uns des autres, mais partageant la même matrice de coefficients, il est plus intéressant de faire cette factorisation une seule fois, et de résoudre les systèmes ensuite.

En effet, l'algorithme de Gauss fournit une matrice triangulaire supérieure U et un vecteur b' tels que (U, b') et (A, b) sont équivalents. Il est possible de voir que toutes les étapes exécutées pendant cette élimination sont en réalité des opérations linéaires dont la matrice associée est triangulaire inférieure atomique. L'inverse de cette matrice est elle-même triangulaire inférieure et ses entrées ne sont autres que les coefficients $m_{i,j}$. Dès lors, si nous appelons cette matrice L , nous avons le système $LUx = b$, qui peut être résolu en résolvant d'abord le système $Ly = b$ et puis le système $Ux = y$.

De manière générale, une factorisation LU d'une matrice A consiste en le fait de déterminer deux matrices L et U (respectivement triangulaire inférieure et triangulaire supérieure) telles que $A = LU$, i.e. :

$$\begin{bmatrix} \ell_{11} & 0 & \dots & 0 \\ \ell_{21} & \ell_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{n1} & \ell_{n2} & \dots & \ell_{nn} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & \dots & u_{1n} \\ 0 & u_{22} & \dots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & u_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}$$

Les matrices L et U ont chacune $\binom{n+1}{2} = \frac{n(n+1)}{2}$ variables libres, et A a n^2 entrées. Intuitivement pour trouver une solution unique à ces n^2 équations à $2\binom{n+1}{2} = n(n+1) = n^2 + n$ inconnues, il faut en fixer n .

L'approche de Doolittle est de fixer les entrées diagonales de L à 1 (i.e. $\ell_{ii} = 1$ pour $1 \leq i \leq n$), et l'approche de Crout est de procéder de la même manière mais sur U (i.e. $u_{ii} = 1$ pour $1 \leq i \leq n$).

L'algorithme de Doolittle utilise donc l'équation suivante :

$$a_{i,j} = (A)_{i,j} = (LU)_{i,j} = \sum_{k=1}^n \ell_{i,k} u_{k,j} = \sum_{k=1}^{\min\{i,j\}} \ell_{i,k} u_{k,j},$$

où la dernière égalité vient du fait que $\ell_{i,k} = 0$ si $k > i$ et $u_{k,j} = 0$ si $k > j$. Il faut utiliser ces égalités dans le bon ordre afin de trouver les coefficients :

- Pour $k \in \{1, \dots, n\}$:
- Calculer la ligne k de U via l'équation :

$$u_{k,j} = a_{k,j} - \sum_{t=1}^{k-1} \ell_{k,t} u_{t,j}$$

- Calculer la colonne k de L via l'équation :

$$\ell_{i,k} = \frac{1}{u_{k,k}} \left[a_{i,k} - \sum_{t=1}^{k-1} \ell_{i,t} u_{t,k} \right]$$

Factorisation de Cholesky

Une matrice $A \in \mathbb{R}^{n \times n}$ est dite *définie positive* (resp. *semi définie positive*) si pour tout vecteur $v \in \mathbb{R}^n$, l'inégalité suivante est vérifiée : $v^\top A v > 0$ (resp. $v^\top A v \geq 0$).

Théorème 3. Une matrice $A \in \mathbb{R}^{n \times n}$ est symétrique et définie positive (resp. semidéfinie positive) si et seulement si elle admet une décomposition $H^\top H$, où H est une matrice triangulaire supérieure dont tous les éléments diagonaux sont strictement positifs (resp. positifs).

À nouveau, cette décomposition permet de résoudre les systèmes linéaires $Ax = b$ en utilisant le fait que $A = H^\top H$. Il faut donc d'abord résoudre $H^\top y = b$ et ensuite $Hx = y$.

La décomposition peut se faire de la manière suivante :

- Pour $i \in \{1, \dots, n\}$:
- Calculer $h_{i,i}$ via l'équation :

$$h_{i,i} = \sqrt{a_{i,i} - \sum_{k=1}^{i-1} h_{k,i}^2}$$

- calculer le reste de la i ème ligne via l'équation (pour $j > i$) :

$$h_{i,j} = \frac{1}{h_{i,i}} \left[a_{i,j} - \sum_{k=1}^{i-1} h_{k,i} h_{k,j} \right].$$

Algorithmes de Jacobi et de Gauss-Seidel

Les algorithmes itératifs ont pour objectif de partir d'une solution initiale $x^{(0)}$ et de la raffiner itérativement. L'objectif est bien sûr d'arriver à une solution au problème posé. Les méthodes linéaires se basent sur $x^{(0)}$ et calculent itérativement les vecteurs $x^{(k+1)}$ satisfaisant une relation sous la forme :

$$Px^{(k+1)} = Nx^{(k)} + b,$$

où $P, N \in \mathbb{R}^{n \times n}$ sont des matrices de coefficients qui dépendent de A .

La méthode de Jacobi consiste à résoudre :

$$Dx^{(k+1)} = (D - A)x^{(k)} + b,$$

pour $D = \text{diag}(A)$, ce qui peut se faire itérativement par la formule (pour $i \in \{1, \dots, n\}$) :

$$x_i^{(k+1)} = \frac{1}{a_{i,i}} \left[b_i - \sum_{j \neq i} a_{i,j} x_j^{(k)} \right].$$

La méthode de Gauss-Seidel part de la même idée mais utilise les *meilleures* estimations $x_j^{(k+1)}$ à la place de $x_j^{(k)}$ si celles-ci ont déjà été calculées :

$$x_i^{(k+1)} = \frac{1}{a_{i,i}} \left[b_i - \sum_{j=1}^{i-1} a_{i,j} x_j^{(k+1)} - \sum_{j=i+1}^n a_{i,j} x_j^{(k)} \right].$$

Ceci correspond bien au formalisme présenté ci-dessus puisque :

$$(D - E)x^{(k+1)} = Fx^{(k)} + b,$$

où E est la négation de la matrice strictement triangulaire inférieure de A alors que F est la négation de la matrice strictement triangulaire supérieure de A (qui satisfont donc l'égalité $D - A = E + F$).

Conditionnement

Le p -conditionnement d'une matrice A est donné par le produit :

$$\kappa_p(A) := \|A\|_p \cdot \|A^{-1}\|_p,$$

où :

$$\|A\|_p := \sup_{\|x\|_p=1} \|Ax\|_p$$

pour une matrice A et :

$$\|x\|_p := \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}$$

pour un vecteur x .

Notez que :

$$\begin{aligned} \|A\|_1 &= \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{i,j}| \\ \|A\|_\infty &= \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{i,j}| = \|A^\top\|_1. \end{aligned}$$

Notez également que si $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ est une matrice inversible, alors son inverse est donnée par :

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

En effet :

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} = \begin{bmatrix} ad - bc & -ab + ab \\ -bc + bc & -bc + ad \end{bmatrix} = (ad - bc) \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Exercice 6.1. Réutilisez le système :

$$\begin{bmatrix} 4 & 8 & 12 \\ 3 & 8 & 13 \\ 2 & 9 & 18 \end{bmatrix} x = \begin{bmatrix} 4 \\ 5 \\ 11 \end{bmatrix},$$

et donnez sa décomposition LU en utilisant la résolution du système par Gauss du TP précédent.

Exercice 6.2. Résolvez le système suivant avec la méthode de Doolittle :

$$\begin{bmatrix} 4 & 3 & 2 & 1 \\ 3 & 4 & 3 & 2 \\ 2 & 3 & 4 & 3 \\ 1 & 2 & 3 & 4 \end{bmatrix} x = \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}$$

Exercice 6.3. Résolvez le système suivant avec la méthode de Cholesky :

$$\begin{bmatrix} 3 & 1 & 0 & 0 \\ 1 & 3 & 1 & 0 \\ 0 & 1 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix} x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Exercice 6.4. Soit le système suivant :

$$\begin{bmatrix} 7 & 10 \\ 5 & 7 \end{bmatrix} x = b.$$

Résolvez-le pour $b = [1, 0.7]^\top$ et pour $b = [1.01, 0.69]^\top$. Qu'en déduisez-vous du conditionnement de la matrice ? Que valent κ_1 et κ_∞ ?

Exercice 6.5. Estimez une solution au système suivant avec (1) la méthode de Jacobi et (2) la méthode de Gauss-Seidel en prenant $x^{(0)} = [0, 0, 0]^\top$ comme solution initiale en effectuant 3 itérations :

$$\begin{bmatrix} 10 & -2 & 1 \\ -2 & 10 & -2 \\ -2 & -5 & 10 \end{bmatrix} x = \begin{bmatrix} 9 \\ 12 \\ 18 \end{bmatrix}.$$

Séance 7 — Équations différentielles ordinaires

Rappel théorique

Une *équation différentielle ordinaire* (EDO ou encore ODE en anglais) est une équation sous la forme :

$$F(t, y, y', y'', \dots, y^{(k)}) = 0,$$

où $y : I \rightarrow \mathbb{R}^d$ est la fonction inconnue. k est l'ordre de l'EDO. Nous nous concentrerons sur les EDOs d'ordre 1.

S'il y a une condition initiale $y(t_0) = y_0$, on parle de *problème de Cauchy*, i.e. :

$$\begin{cases} y'(t) &= f(t, y(t)) \\ y(t_0) &= y_0 \end{cases}$$

Une fonction $f : \Omega \subset \mathbb{R} \rightarrow \mathbb{R}$ est dite *Lipschitz-continue* de constante $L > 0$ si pour toute paire $(x, y) \in \Omega^2$, l'inégalité suivante est vérifiée :

$$|f(x) - f(y)| \leq L |x - y|.$$

La fonction $f : I \times \mathbb{R} \rightarrow \mathbb{R}$ est *Lipschitz-continue en espace* s'il existe un $L > 0$ tel que pour tout $t \in I$, la fonction $f_t : \mathbb{R} \rightarrow \mathbb{R} : y \mapsto f(t, y)$ est L -Lipschitz-continue. La fonction doit donc être Lipschitz de manière "uniforme" puisque cette constante L doit être la même pour toute valeur de t .

Théorème 4 (Cauchy-Lipschitz "fort"). Si $f : I \times \mathbb{R} \rightarrow \mathbb{R}$ est continue et L -Lipschitz-continue pour un certain $L > 0$, alors le problème de Cauchy suivant admet une unique solution $y \in \mathcal{C}^1(I, \mathbb{R})$:

$$\begin{cases} y'(t) &= f(t, y(t)) \\ y(t_0) &= y_0 \end{cases}$$

De plus, cette solution est donnée par :

$$y : I \rightarrow \mathbb{R} : t \mapsto y_0 + \int_{t_0}^t f(s, y(s)) ds.$$

Proposition 5. Si $f \in \mathcal{C}^1(I \times \mathbb{R})$ et $\frac{\partial f}{\partial y}$ est bornée en module, alors f est Lipschitz-continue. En particulier, si I est un intervalle fermé et $f \in \mathcal{C}^1(I \times \mathbb{R}, \mathbb{R})$, alors f est Lipschitz-continue.

Corollaire 6. Soit $f \in \mathcal{C}^1(I \times \mathbb{R}, \mathbb{R})$. f est L -Lipschitz-continue si et seulement si $\frac{\partial f}{\partial y}$ est bornée par L en module.

Approximations de Taylor

Théorème 7 (Taylor). Si $f : \mathbb{R} \rightarrow \mathbb{R}$ est k fois dérivable en $a \in \mathbb{R}$, alors il existe une fonction $R_{a,k} : \mathbb{R} \rightarrow \mathbb{R}$ telle que :

$$f(a + h) = \sum_{\ell=0}^k \frac{f^{(\ell)}(a)}{\ell!} h^\ell + R_{a,k}(h),$$

où $R_{a,k}(h) = o(h^k)$ (pour $h \rightarrow 0$).

En particulier, ce théorème nous dit que la meilleure approximation locale de la fonction f autour de a par un polynôme de degré k (où f est k fois dérivable en a) est donnée par le polynôme $\sum_{\ell=0}^k \beta_{\ell}(x-a)^{\ell}$ où les coefficients β_{ℓ} sont donnés par $\beta_{\ell} = \frac{1}{\ell!} f^{(\ell)}(a)$.

Nous allons utiliser ce principe pour avoir une forme fermée qui va nous servir d'approximation de la solution y au problème de Cauchy donné. En particulier, nous dirons que (au moins pour t proche de t_0) :

$$y(t) \approx \hat{y}(t) := \sum_{\ell=0}^k \frac{y^{(\ell)}(t_0)}{\ell!} (t - t_0)^{\ell}.$$

En particulier, nous n'avons besoin d'évaluer y et ses k premières dérivées qu'au point t_0 , ce que nous pouvons faire en utilisant la fonction f . En effet, nous savons que :

$$\begin{aligned} y^{(0)}(t_0) &= y(t_0) = y_0 \\ y^{(\ell+1)}(t_0) &= \left. \frac{d^{\ell}}{dt^{\ell}} f(t, y(t)) \right|_{t=t_0} \end{aligned}$$

L'approximation $\hat{y}$ est donc forcément un polynôme.

Méthodes itératives d'Euler

Une approximation de Taylor locale d'ordre 1 de la fonction y autour de t_0 nous donne (pour $h = t - t_0$) :

$$y(t) \approx y(t_0) + hy'(t_0) = y(t_0) + h \cdot f(t_0, y(t_0)).$$

Sur base de cette approximation, nous allons approximer la solution au problème de Cauchy en déterminant un vecteur de valeurs $\hat{y}(t_n)$ pour $0 \leq n \leq N$ (où $t_0 < t_1 < \dots < t_N$), où (normalement) $\hat{y}(t_n) \approx y(t_n)$. Nous allons plus précisément choisir $t_n = t_0 + n \cdot h$, où $h > 0$ est le pas qui est fixé.

La méthode *progressive* d'Euler est une approche explicite où nous définissons :

$$\begin{aligned} \hat{y}(t_0) &:= y_0 \\ \hat{y}(t_{n+1}) &:= \hat{y}(t_n) + h \cdot f(t_n, \hat{y}(t_n)). \end{aligned}$$

La méthode *rétrograde* d'Euler est une approche implicite qui ne donne pas une forme fermée pour déterminer les valeurs $\hat{y}(t_n)$. À la place, nous disons que pour $n > 0$, $\hat{y}(t_n)$ est solution au problème :

$$\hat{y}(t_n) = \hat{y}(t_{n-1}) + h \cdot f(t_n, \hat{y}(t_n)).$$

Exercice 7.1. Soient les deux fonctions suivantes :

$$f_1(t, y) := t^2 \cos^2 y + t \sin^2 y \quad \text{et} \quad f_2(t, y) := \sqrt{|y|}.$$

Montrez que f_1 est 4-Lipschitz-continue si $|t| < 1$ et montrez que f_2 n'est pas Lipschitz-continue.

Exercice 7.2. Approximez la solution du problème de Cauchy suivant autour de 0 à l'aide d'une approximation de Taylor d'ordre 3 :

$$\begin{cases} y'(t) &= f(t, y(t)) := t + y(t) \\ y(0) &= 2 \end{cases}$$

Exercice 7.3. Reprenez le même système de Cauchy et calculez les 4 premières approximations $\hat{y}(t_n)$ avec la méthode d'Euler progressive (pour $h = 0.1$). Calculez l'erreur absolue de l'approximation en sachant que la solution exacte est $y(t) = 3 \exp(t) - (t + 1)$.

Exercice 7.4. Résolvez le problème de Cauchy suivant par la méthode d'Euler progressive :

$$\begin{cases} y'(t) &= t^2 - y(t) \\ y(0) &= 1 \end{cases}$$

1. Utilisez $h = 0.2$ et faites deux itérations.
2. Utilisez $h = 0.1$ et faites 4 itérations.
3. Comparez avec la solution exacte $y(t) = -\exp(-t) + t^2 - 2t + 2$.

Exercice 7.5. Résolvez le problème de Cauchy suivant sur l'intervalle $[0, 0.5]$ avec la méthode d'Euler rétrograde pour $h = 0.1$:

$$\begin{cases} y'(t) &= t(y(t) + 1) \\ y(0) &= 0. \end{cases}$$

Séance 8 — Interpolation

Rappel théorique

Le principe de l'*interpolation* est de déterminer une fonction f de forme connue qui va passer par un ensemble de points donnés. Plus formellement, pour deux ensembles $X \subseteq \mathbb{R}^p$ et $Y \subseteq \mathbb{R}^q$ et pour $((x_i, y_i))_{i=1}^n$ où $x_i \in X$ et $y_i \in Y$, nous voulons une fonction $f : X \rightarrow Y$ satisfaisant $y_i = f(x_i)$ pour toute valeur de i . Ici, nous considérerons $X = Y = \mathbb{R}$.

Interpolation polynomiale

Théorème 8. Soient $n+1$ paires (x_i, y_i) ($0 \leq i \leq n$) telles que $x_i \neq x_j$ pour tous $i \neq j$. Il existe un unique polynôme Π_n de degré au plus n tel que $\Pi_n(x_i) = y_i$ pour tout $0 \leq i \leq n$.

Trouver ce polynôme peut se formuler comme la résolution du système linéaire suivant :

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \beta = \mathbf{y},$$

où $\mathbf{y}$ est le vecteur $[y_0, y_1, \dots, y_n]^\top$ et β est le vecteur de coefficients β_i satisfaisant $\Pi_n : x \mapsto \sum_{k=0}^n \beta_k x^k$. Cependant, la matrice (appelée *matrice de Vandermonde*) est très mal conditionnée (intuitivement, c'est dû au fait que le rapport entre la plus grande et la plus petite valeur dans cette matrice – en module – est très grande à cause de l'exponentiation). De plus, le nombre d'opérations nécessaires pour résoudre un système linéaire $n \times n$ est cubique en n .

Dès lors il est intéressant de résoudre ce problème différemment.

Interpolation de Lagrange

Les *polynômes de Lagrange* associés aux couples (x_i, y_i) sont les polynômes (tous d'ordre n) :

$$\ell_i : \mathbb{R} \rightarrow \mathbb{R} : x \mapsto \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

Ces polynômes ont les propriétés suivantes (pour tous $i \neq j$) :

- $\ell_i(x_i) = 1$;
- $\ell_i(x_j) = 0$.

Dès lors nous pouvons définir le polynôme Π_n comme suit :

$$\Pi_n(x) := \sum_{i=0}^n y_i \cdot \ell_i(x).$$

Ce polynôme satisfait bien, par définition $\Pi_n(x_k) = x_k$ pour tout k puisque :

$$\Pi_n(x_k) = \sum_{i=0}^n y_i \cdot \ell_i(x_k) = \sum_{i \neq k} y_i \cdot \ell_i(x_k) + y_k \cdot \ell_k(x_k) = 0 + y_k \cdot 1 = y_k.$$

Déterminer les coefficients de ℓ_i se fait en temps $\Theta(n)$ pour tout i , dès lors le nombre d'opérations totales est quadratique en n .

Interpolation de Newton

Le polynôme suivant est appelé le *mème polynôme nodal* (et a degré m) :

$$\omega_m(x) = \prod_{i=0}^{m-1} (x - x_i).$$

Le polynôme ω_{n+1} est appelé le *polynôme nodal*.

Remarque. Avec cette notation, le polynôme de Lagrange $\ell_i(x)$ peut se réécrire comme suit :

$$\ell_i(x) = \frac{\omega_{n+1}(x)}{(x - x_i)\omega'_{n+1}(x_i)}. \quad (1)$$

En effet :

$$\begin{aligned} \frac{\omega_{n+1}(x)}{(x - x_i)} &= \prod_{j \neq i} (x - x_j), \\ \omega'_{n+1}(x) &= \frac{d \omega_{n+1}(x)}{d x} = \sum_{j=0}^n \frac{d(x - x_j)}{d x} \prod_{k \neq j} (x_i - x_k) = \sum_{j=0}^n \prod_{k \neq j} (x - x_k). \end{aligned}$$

En particulier $\omega'_{n+1}(x_i) = \sum_{j \neq i} \prod_{k \neq j} (x_i - x_k) + \prod_{k \neq i} (x_i - x_k) = \prod_{j \neq i} (x_i - x_j)$.

La méthode de Newton pour l'interpolation polynômiale consiste en le fait de construire itérativement les polynômes $\Pi_1, \dots$ jusqu'à Π_n , où la détermination du polynôme Π_{m+1} se fait sur la base du polynôme Π_m (utilisant les m premiers points) et du point (x_m, y_m) . Notons $q_{m+1}(x) := \Pi_{m+1}(x) - \Pi_m(x)$, et réalisons que (1) Π_m et Π_{m+1} s'annulent tous deux en les valeurs x_i pour $0 \leq i \leq m$ et (2) q_{m+1} est la différence entre un polynôme de degré au plus $m + 1$ et d'un polynôme de degré au plus m , et donc $\deg q_{m+1} \leq m + 1$ également. Dès lors nous savons que :

$$q_{m+1}(x) = a_{m+1}(x - x_0)(x - x_1) \dots (x - x_m) = a_{m+1}\omega_{m+1}(x),$$

pour un certain $a_{m+1} \in \mathbb{R}$. Dès lors, nous savons que :

$$a_{m+1} = \frac{q_{m+1}(x_{m+1})}{\omega_{m+1}(x_{m+1})} = \frac{\Pi_{m+1}(x_{m+1}) - \Pi_m(x_{m+1})}{\omega_{m+1}(x_{m+1})} = \frac{f(x_{m+1}) - \Pi_m(x_{m+1})}{\omega_{m+1}(x_{m+1})}.$$

Cette quantité est appelée *différence divisée de Newton* et est notée $f[x_0, \dots, x_{m+1}]$, ou encore $D^{m+1}f$. Nous avons alors que :

$$\begin{aligned} \Pi_n(x) &= q_n(x) + \Pi_{n-1}(x) = f[x_0, \dots, x_n]\omega_n(x) + \Pi_{n-1}(x) \\ &= f[x_0, \dots, x_n]\omega_n(x) + f[x_0, \dots, x_{n-1}]\omega_{n-1}(x) + \Pi_{n-2}(x) \\ &= \dots \\ &= \sum_{m=0}^n \omega_m(x) f[x_0, \dots, x_m]. \end{aligned}$$

Les différences divisées sont trouvées par la relation suivante :

$$f[x_i, \dots, x_{i+k}] = \begin{cases} f(x_i) = y_i & \text{si } k = 0 \\ \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i} & \text{si } k \geq 1. \end{cases}$$

Démonstration. Pour montrer que cette relation de récurrence sur les différences divisées permet bien de reconstruire le polynôme interpolant, montrons quelque chose d'un peu plus fort. Pour le justifier, reprenons l'équation 1 qui nous dit que pour tout $1 \leq m \leq n+1$:

$$\Pi_m(x) = \sum_{i=0}^m \frac{y_i}{\omega'_{m+1}(x_i)} \frac{\omega_{m+1}(x)}{x - x_i},$$

dès lors, si $[x^k]\Pi_m$ désigne le coefficient de x^k dans le polynôme Π_m , nous avons :

$$f[x_0, \dots, x_m] = [x^m]\Pi_m = \sum_{i=0}^m \frac{y_i}{\omega'_{m+1}(x_i)} = \sum_{i=0}^m y_i \prod_{\substack{j=0 \\ j \neq i}}^m \frac{1}{(x_i - x_j)}.$$

Montrons alors que pour tous $0 \leq j \leq k \leq n$:

$$f[x_j, \dots, x_k] = \sum_{i=j}^k y_i \prod_{\substack{\ell=j \\ \ell \neq i}}^k \frac{1}{(x_i - x_\ell)}. \quad (2)$$

Pour cela, procédons par induction sur $k - j$.

Quand $k = j$, alors :

$$\sum_{i=j}^j y_i \prod_{\ell \in \emptyset} (x_i - x_\ell) = y_j = f[x_j].$$

Maintenant supposons que l'équation 2 est vérifiée pour toute paire (j', k') telle que $0 \leq j' \leq k' \leq m$ et $k' - j' < k - j$, et montrons qu'elle reste vraie pour (j, k) . Commençons par déterminer $f[x_{j+1}, \dots, x_k] - f[x_j, \dots, x_{k-1}]$. Pour cela, appliquons l'hypothèse d'induction afin d'obtenir :

$$\begin{aligned} f[x_{j+1}, \dots, x_k] - f[x_j, \dots, x_{k-1}] &= \sum_{i=j+1}^k y_i \prod_{\substack{\ell=j+1 \\ \ell \neq i}}^k \frac{1}{(x_i - x_\ell)} - \sum_{i=j}^{k-1} y_i \prod_{\substack{\ell=j \\ \ell \neq i}}^{k-1} \frac{1}{(x_i - x_\ell)} \\ &= y_k \prod_{\ell=j+1}^{k-1} \frac{1}{(x_k - x_\ell)} + \sum_{i=j+1}^{k-1} y_i \left(\prod_{\substack{\ell=j+1 \\ \ell \neq i}}^k \frac{1}{(x_i - x_\ell)} - \prod_{\substack{\ell=j \\ \ell \neq i}}^{k-1} \frac{1}{(x_i - x_\ell)} \right) - y_j \prod_{\ell=j+1}^k \frac{1}{(x_j - x_\ell)} \\ &= y_k \prod_{\ell=j+1}^{k-1} \frac{1}{(x_k - x_\ell)} + \sum_{i=j+1}^{k-1} y_i \left(\prod_{\substack{\ell=j+1 \\ \ell \neq i}}^{k-1} \frac{1}{(x_i - x_\ell)} \right) \left(\frac{1}{(x_i - x_k)} - \frac{1}{(x_i - x_j)} \right) - y_j \prod_{\ell=j+1}^k \frac{1}{(x_j - x_\ell)} \\ &= y_k \prod_{\ell=j+1}^{k-1} \frac{1}{(x_k - x_\ell)} + \sum_{i=j+1}^{k-1} y_i \left(\prod_{\substack{\ell=j+1 \\ \ell \neq i}}^{k-1} \frac{1}{(x_i - x_\ell)} \right) \frac{(x_k - x_j)}{(x_i - x_k)(x_i - x_j)} - y_j \prod_{\ell=j+1}^k \frac{1}{(x_j - x_\ell)} \\ &= y_k \prod_{\ell=j+1}^{k-1} \frac{1}{(x_k - x_\ell)} + (x_k - x_j) \sum_{i=j+1}^{k-1} y_i \prod_{\substack{\ell=j \\ \ell \neq i}}^k \frac{1}{(x_i - x_\ell)} - y_j \prod_{\ell=j+1}^k \frac{1}{(x_j - x_\ell)} \end{aligned}$$

$$\begin{aligned}
&= (x_k - x_j) y_k \prod_{\substack{\ell=j \\ \ell \neq k}}^k \frac{1}{(x_k - x_\ell)} + (x_k - x_j) \sum_{i=j+1}^{k-1} y_i \prod_{\substack{\ell=j \\ \ell \neq i}}^k \frac{1}{(x_i - x_\ell)} - (x_j - x_k) y_j \prod_{\substack{\ell=j \\ \ell \neq j}}^k \frac{1}{(x_j - x_\ell)} \\
&= (x_k - x_j) y_k \prod_{\substack{\ell=j \\ \ell \neq k}}^k \frac{1}{(x_k - x_\ell)} + (x_k - x_j) \sum_{i=j+1}^{k-1} y_i \prod_{\substack{\ell=j \\ \ell \neq i}}^k \frac{1}{(x_i - x_\ell)} + (x_k - x_j) y_j \prod_{\substack{\ell=j \\ \ell \neq j}}^k \frac{1}{(x_j - x_\ell)} \\
&= (x_k - x_j) \sum_{i=j}^k y_i \prod_{\substack{\ell=j \\ \ell \neq i}}^k \frac{1}{(x_i - x_\ell)}.
\end{aligned}$$

Dès lors, nous avons bien que :

$$f[x_j, \dots, x_k] = \frac{f[x_{j+1}, \dots, x_k] - f[x_j, \dots, x_{k-1}]}{x_k - x_j} = \sum_{i=j}^k y_i \prod_{\substack{\ell=j \\ \ell \neq i}}^k \frac{1}{(x_i - x_\ell)},$$

comme voulu dans l'équation 2.

Par induction, nous pouvons en déduire que $\sum_{t=0}^m f[x_0, \dots, x_t] \omega_t(x) = \Pi_{m+1}(x)$, et donc que la forme générale de polynôme interpolant de Newton est bien celui recherché. $\square$

Nous pouvons également définir le vecteur $D^k f$ qui contient toutes les différences divisées à k donné.

En pratique on calcule d'abord toutes les différences d'ordre 1, puis celles d'ordre 2, etc. Si on représente ces quantités dans une matrice triangulaire inférieure, on obtient les coefficients nécessaires dans la diagonale. Le nombre d'opérations nécessaires est donc à nouveau quadratique en n .

Remarque : les différences divisées sont en quelque sorte une généralisation discrète des dérivées. En effet : tout comme la dérivée d'un polynôme de degré $\leq n$ est un polynôme de degré $\leq n - 1$, l'opérateur de dérivée discrète Δ appliqué à un polynôme de degré $\leq n$ donne un polynôme de degré $\leq n - 1$:

$$\Delta \left(\sum_{k=0}^n a_k x^k \right) = \sum_{k=0}^n a_k (x+1)^k - \sum_{k=0}^n a_k x^k = \sum_{k=0}^n a_k \left(\sum_{j=0}^{k-1} \binom{k}{j} x^j \right) = \sum_{j=0}^{n-1} \left(\sum_{k=j+1}^n a_k \binom{k}{j} \right) x^j.$$

En particulier, appliquer itérativement ces différences (normalisées ou non) sur un polynôme (ayant donc degré fini) finira nécessairement par donner identiquement 0.

8.1 Interpolation de Lagrange

Exercice 8.1. Déterminez le polynôme de degré minimal passant par les points $(0, -2)$, $(-1, -2)$ et $(\frac{1}{2}, -\frac{5}{4})$ en utilisant la méthode de Lagrange.

Exercice 8.2. Donnez une approximation de la valeur $\sqrt[3]{20}$ en utilisant l'interpolation de Lagrange.

8.2 Interpolation de Newton

Exercice 8.3. Déterminez le polynôme de Newton passant par les points $(6, 6)$, $(8, 24)$, $(10, 60)$ et $(12, 120)$.

Exercice 8.4. Donnez une forme fermée pour la fonction $\varphi : \mathbb{N} \rightarrow \mathbb{R} : n \mapsto \sum_{k=0}^n k^2$ en utilisant la méthode de Newton.

Hint : Nous savons que φ est en réalité un polynôme. Quel est son degré ? Et en avons-nous besoin pour commencer ?

Exercice 8.5 (Pas fait en séance). Soit la fonction $f(x) = \sin x$. Considérons les valeurs $x = [0, 0.2, 0.4, 0.6, 0.8]$.

1. Donnez une borne supérieure pour l'erreur d'approximation de f en $x = 0.28$.
2. Comparez cette borne avec l'erreur réelle en sachant que $\sin(0.28) \approx 0.276\,355\,665$.

Séance 9 — Interpolation (partie 2)

Rappel théorique

L'interpolation est un problème qui n'est pas stable, tout particulièrement lorsque les x_i sont équirépartis dans un intervalle $[a, b]$. Pour éviter ce problème, nous pouvons interpoler *localement* la fonction en sélectionnant itérativement un sous-ensemble de points *successifs* et en interpolant un polynôme entre ces points-là uniquement.

L'exemple de base est l'interpolation linéaire par morceaux : entre chaque paire de points consécutifs, nous déterminons l'unique droite qui passe par ces deux points. Cependant, cette fonction a beau être continue, elle n'est (*a priori*) pas dérivable en les points x_i .

Une *spline* d'ordre k pour les points $x_0, \dots, x_n$ est une fonction $s \in \mathcal{C}^{k-1}([a, b], \mathbb{R})$ telle que $s|_{[x_i, x_{i+1}]}$ est un polynôme de degré k . Notons $s_i := s|_{[x_i, x_{i+1}]}$. s peut donc être vue comme n polynômes de degré k collés les uns après les autres. Dès lors s est définie par $n(k+1)$ coefficients. Intuitivement, il va nous falloir autant de contraintes pour pouvoir déterminer s de manière unique. Nous imposons donc :

1. $s(x_i) = f(x_i)$ pour $0 \leq i \leq n$;
2. s est $k-1$ fois dérivable en les points x_i , donc les $s_i^{(m)}$ doivent se rejoindre en ces points :

$$s_i^{(m)}(x_{i+1}) = s_{i+1}^{(m)}(x_{i+1}) \quad \text{pour } 0 \leq i \leq n-2 \text{ et } 0 \leq m \leq k-1.$$

Ainsi nous avons $n+1 + (n-1)k$ contraintes, il nous en reste $(k+1)n - (n+1) - (n-1)k = k-1$ à déterminer.

- Si nous imposons $s^{(m)}(a) = s^{(m)}(b)$ pour $1 \leq m \leq k-1$, alors nous disons que s est une *spline périodique*.
- Si $k \geq 3$ est impair et que nous imposons $s^{(m)}(a) = s^{(m)}(b) = 0$ pour $\frac{k+1}{2} \leq m \leq k-1$, alors nous disons que s est une *spline naturelle*.

Nous nous concentrerons sur le cas $k=3$ ici (on parle alors de *splines cubiques*). Dans ce cas-ci, introduisons les notations suivantes (pour $0 \leq i \leq n$) :

$$\begin{aligned} y_i &:= s(x_i) \\ m_i &:= s'(x_i) \\ M_i &:= s''(x_i), \end{aligned}$$

et pour $1 \leq i \leq n$, nous notons $h_i := x_i - x_{i-1}$.

Nous savons que tous les s_i sont des polynômes de degré 3, dès lors s_i'' est une droite. De plus, nous savons que $s_i''(x_i) = M_i$ et $s_i''(x_{i+1}) = M_{i+1}$ par continuité de s'' . Dès lors :

$$s_i''(x) = M_i + \frac{M_{i+1} - M_i}{h_{i+1}}(x - x_i).$$

Nous en déduisons que :

$$s_i'(x) = m_i + \int_{x_i}^x s_i''(t) dt = m_i + M_i(x - x_i) + \frac{M_{i+1} - M_i}{2h_{i+1}}(x - x_i)^2,$$

et de plus :

$$s_i(x) = y_i + \int_{x_i}^x s'(t)dt = y_i + m_i(x - x_i) + \frac{M_i}{2}(x - x_i)^2 + \frac{M_{i+1} - M_i}{6h_{i+1}}(x - x_i)^3.$$

Puisque $s_i(x_{i+1}) = y_{i+1}$ par continuité de s , nous pouvons trouver m_i :

$$y_{i+1} = s_i(x_{i+1}) = y_i + m_i h_{i+1} + \frac{M_i}{2} h_{i+1}^2 + \frac{M_{i+1} - M_i}{6} h_{i+1}^3,$$

i.e. :

$$m_i = \frac{y_{i+1} - y_i}{h_{i+1}} - \frac{M_i}{3} h_{i+1} - \frac{M_{i+1} - M_i}{6} h_{i+1}.$$

La continuité de s' nous permet finalement d'exprimer la contrainte suivante (dite de M -continuité) :

$$\begin{aligned} s'_{i-1}(x_i) &= s'(x_i) = s'_i(x_i) \\ m_{i-1} + M_{i-1} h_i + \frac{M_i - M_{i-1}}{2} h_i &= m_i \\ \frac{y_i - y_{i-1}}{h_i} + \frac{M_{i-1}}{6} h_i + \frac{M_i}{3} h_i &= \frac{y_{i+1} - y_i}{h_{i+1}} - \frac{M_i}{3} h_{i+1} - \frac{M_{i+1} - M_i}{6} h_{i+1} \\ \frac{M_{i-1} h_i}{6} + \frac{M_i(h_i + h_{i+1})}{3} + \frac{M_{i+1} h_{i+1}}{6} &= \frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i} \end{aligned}$$

En particulier si $h_i = h$ pour tout i , alors ces équations peuvent être simplifiées en :

$$\frac{h^2}{6} M_{i-1} + \frac{2h^2}{3} M_i + \frac{h^2}{6} M_{i+1} = y_{i+1} + y_{i-1} - 2y_i.$$

Maintenant, si s est une spline naturelle, il reste à ajouter les contraintes $M_0 = 0$ et $M_n = 0$. Si par contre s est une spline périodique, il faut ajouter les contraintes $M_0 = M_n$ et $s'(x_0) = s'(x_n)$:

$$\frac{y_1 - y_0}{h_1} - \frac{M_0 h_1}{3} - \frac{M_1 h_1}{6} = s'_0(x_0) = s'_{n-1}(x_n) = \frac{y_n - y_{n-1}}{h_n} + \frac{M_{n-1} h_n}{6} + \frac{M_n h_n}{3},$$

ou encore :

$$\frac{y_1 - y_0}{h_1} - \frac{y_n - y_{n-1}}{h_n} = \frac{M_0 h_1}{3} + \frac{M_1 h_1}{6} + \frac{M_{n-1} h_n}{6} + \frac{M_n h_n}{3}.$$

Curve fitting

Le problème de *curve fitting* peut se formuler comme suit : pour (x_i, y_i) ($0 \leq i \leq n$) une séquence de points, quelle fonction h correspond au mieux à ces points ?

Pour définir tout ceci formellement, il faut définir quel est l'ensemble des fonctions h considérées et définir ce que veut dire *correspondre* ici. Nous noterons $\mathcal{H}$ l'ensemble des fonctions h considérées. Pour une fonction $h \in \mathcal{H}$, nous définissons l'erreur de h sur le point (x_i, y_i) par :

$$\mathcal{L}(x_i, y_i; h) := \|h(x_i) - y_i\|_2^2.$$

L'erreur sur l'ensemble des points est donné par :

$$\mathcal{L}(\mathbf{x}, \mathbf{y}; h) := \sum_{i=0}^n \mathcal{L}(x_i, y_i; h),$$

où $\mathbf{x}$ et $\mathbf{y}$ sont les vecteur $\mathbf{x} = [x_0, \dots, x_n]^\top$ et $\mathbf{y} = [y_0, \dots, y_n]^\top$.

Notez que dans le cas paramétrique (toutes les fonctions $h \in \mathcal{H}$ ont la même forme mais sont paramétrisées par un certain $\theta \in \Theta$), nous noterons $h_\theta \in \mathcal{H}$ pour expliciter la dépendance en θ .

Le problème de *curve fitting* consiste donc à trouver la fonction $h \in \mathcal{H}$ qui minimise $\mathcal{L}(\mathbf{x}, \mathbf{y}; h)$.

Restreignons-nous maintenant au cas où $\mathcal{H}$ est l'ensemble des polynômes de degré au plus m (à m fixé). Ici, $\Theta = \mathbb{R}^{m+1}$ et pour $\theta \in \Theta$, la fonction h_θ est définie par :

$$h_\theta(x) = \sum_{j=0}^m \theta_j x^j.$$

Nous cherchons donc à résoudre le problème :

$$\theta^* = \arg \min_{\theta \in \Theta} \mathcal{L}(\mathbf{x}, \mathbf{y}; h_\theta).$$

Nous savons qu'une telle solution doit satisfaire :

$$\nabla_\theta \mathcal{L}(\mathbf{x}, \mathbf{y}; h_\theta) = 0.$$

Notons également que, comme observé précédemment, puisque les x_i sont constants (par rapport à θ), nous pouvons voir le vecteur $h_\theta(\mathbf{x}) = [h_\theta(x_0), \dots, h_\theta(x_n)]^\top$ comme le produit :

$$h_\theta(\mathbf{x}) = V(\mathbf{x})\theta,$$

où $V(\mathbf{x})$ est la matrice de Vandermonde définie par $V(\mathbf{x})_{i,j} = x_i^j$. Nous voulons alors résoudre le système (où $0 \leq k \leq m$) :

$$0 = \frac{\partial}{\partial \theta_k} \mathcal{L}(\mathbf{x}, \mathbf{y}; h_\theta) = \sum_{i=0}^n \frac{\partial}{\partial \theta_k} (h_\theta(x_i) - y_i)^2 = 2 \sum_{i=0}^n (h_\theta(x_i) - y_i) \frac{\partial h_\theta(x_i)}{\partial \theta_k}.$$

Or dans notre cas :

$$\frac{\partial}{\partial \theta_k} h_\theta(x) = \sum_{j=0}^m \frac{\partial \theta_j}{\partial \theta_k} x^j = x^k.$$

Dès lors, le système à résoudre est donc (pour tout k) :

$$0 = \sum_{i=0}^n (h_\theta(x_i) - y_i) x_i^k = \sum_{i=0}^n ((V(\mathbf{x})\theta)_i - y_i) V(\mathbf{x})_{i,k} = \left(V(\mathbf{x})^\top (V(\mathbf{x})\theta - \mathbf{y}) \right)_k,$$

et donc de manière vectorielle :

$$V(\mathbf{x})^\top V(\mathbf{x})\theta = V(\mathbf{x})^\top \mathbf{y}.$$

Notez que contrairement au système $V(\mathbf{x})\theta = \mathbf{y}$, qui était surdimensionné, ce système-ci admet bien une unique solution (pour peu que les x_i soient bien deux à deux distincts) puisque la matrice $V(\mathbf{x})^\top V(\mathbf{x})$ est bien carrée (de dimension $(m+1) \times (m+1)$).

Remarque : puisque $V(\mathbf{x})^\top V(\mathbf{x})$ est inversible, cela revient à effectuer le calcul :

$$\theta^* = \underbrace{\left(V(\mathbf{x})^\top V(\mathbf{x}) \right)^{-1}}_{=: V(\mathbf{x})^\dagger} V(\mathbf{x})^\top y,$$

où $V(\mathbf{x})^\dagger$ est la *pseudo-inverse de Moore* de la matrice $V(\mathbf{x})$. Ce système est appelé le *système des équations normales* du problème.

Exercice 9.1. Interpolez les points $(-1, 0)$, $(0, 1)$ et $(1, -1)$ par splines cubiques naturelle et périodique.

Exercice 9.2. Déterminez la spline naturelle cubique passant par les points donnés dans le tableau suivant :

x	$f(x)$
0	2
1	4.4366
$\frac{3}{2}$	6.7134
$\frac{9}{4}$	13.9130

En sachant que $f(x) = 2 \exp(x) - x^2$, évaluez la spline en $x = 0.66$ et $x = 1.75$. Qu'en déduisez-vous ?

Exercice 9.3. Déterminez le polynôme quadratique qui approxime au mieux les points suivants :

$\mathbf{x}$	$\mathbf{y}$
1	-2
3	-0.8
4	-0.5
5	0.1
6	0.7
8	1

Séance 10 — Résolution d'équations non-linéaires

Rappel théorique

Nous voulons déterminer une solution à l'équation $f(x) = 0$ pour une certaine fonction $f : [a, b] \subset \mathbb{R} \rightarrow \mathbb{R}$. De manière équivalente, nous pouvons regarder l'équation $f(x) = d$. Notons que ce problème peut s'exprimer comme $x^* = F(d)$ pour $F(d) = f^{-1}(d)$.

La fonction f peut potentiellement n'avoir aucune racine, peut en avoir une seule, mais peut également en avoir plusieurs. De plus une solution peut être de *multiplicité* $m \geq 1$. Ces différents cas ne sont pas équivalents. En effet, le problème n'est bien posé que si une unique solution existe.

Le conditionnement absolu du problème est alors :

$$\kappa_{\text{abs}}(d) = \|F'(d)\| = \left\| \frac{1}{f'(x^*)} \right\| = \frac{1}{|f'(x^*)|},$$

où x^* satisfait $f(x^*) = d$.

Dès lors, nous en déduisons que le problème n'est bien conditionné que lorsque $|f'(x^*)| \gg 0$. En particulier, si la racine x^* est de multiplicité $m \geq 2$, alors le problème est très mal conditionné.

Méthode de la bisection

Théorème 9 (Théorème des valeurs intermédiaires (IVT)). Soient $f \in \mathcal{C}^0([a, b], \mathbb{R})$ et $y \in \mathbb{R}$. Si $\min\{f(a), f(b)\} \leq y \leq \max\{f(a), f(b)\}$, alors il existe $x \in [a, b]$ tel que $f(x) = y$.

Corollaire 10. Sous les mêmes conditions, si $f(a)f(b) < 0$, alors il existe $x^* \in [a, b]$ tel que $f(x^*) = 0$.

En utilisant ce principe, nous pouvons alors définir les séquences $x^{(k)}, a^{(k)}, b^{(k)}$ comme suit :

$$\begin{aligned} x^{(k)} &= \frac{1}{2} (a^{(k)} + b^{(k)}), \\ a^{(0)} &= a \quad a^{(k)} = \begin{cases} a^{(k-1)} & \text{si } \text{sign}(f(x^{(k-1)})) = \text{sign}(f(b^{(k-1)})), \\ x^{(k-1)} & \text{sinon,} \end{cases} \\ b^{(0)} &= b \quad b^{(k)} = \begin{cases} b^{(k-1)} & \text{si } \text{sign}(f(x^{(k-1)})) = \text{sign}(f(a^{(k-1)})), \\ x^{(k-1)} & \text{sinon.} \end{cases} \end{aligned}$$

Méthodes linéarisées

Notons x^* la solution au problème et supposons que $f \in \mathcal{C}^1([a, b])$. Par le théorème de Taylor d'ordre 1 (avec évaluation de l'erreur) autour de x^* , nous savons que pour tout x , il existe un certain $\xi = \xi(x) \in [a, x^*]$ tel que :

$$f(x) = f(x^*) + f'(\xi)(x - x^*) = f'(\xi)(x - x^*).$$

En particulier :

$$x^* = x - \frac{f(x)}{f'(\xi)}.$$

Dès lors, nous pouvons établir la méthode (explicite) suivante (pour $k \geq 0$) :

$$x^{(k+1)} = x^{(k)} - f(x)q_k^{-1},$$

où $q_k \approx f'(\xi)$.

- Si $q_k = q = \frac{f(b)-f(a)}{b-a}$, alors nous parlons de *méthode de la corde*.
- Si $q_k = \frac{f(x^{(k)})-f(x^{(k-1)})}{x^{(k)}-x^{(k-1)}}$, alors nous parlons de *méthode de la sécante*.
- Si $q_k = \frac{f(x^{(k)})-f(x^{(k')})}{x^{(k)}-x^{(k')}} (où k' est le plus grande entier $< k$ tel que $f(x^{(k)})f(x^{(k')}) < 0$), alors nous parlons de *méthode de la fausse position*.$
- Si $q_k = f'(x^{(k)})$, alors nous parlons de la méthode de Newton-Raphson.

La méthode de la fausse position admet une convergence globale, alors que les trois autres convergent localement, c-à-d qu'il existe un certain $\delta > 0$ tel que si $|x^{(0)} - x^*| < \delta$, alors $x^{(k)} \xrightarrow[k \rightarrow +\infty]{} x^*$.

Définition 1. Considérons une suite $(x^{(k)})_k$ convergente vers x^* . S'il existe $C, k_0 > 0$ et $p \geq 1$ tels que pour tout $k \geq k_0$:

$$|x^{(k+1)} - x^*| < C |x^{(k)} - x^*|^p,$$

alors nous disons que la convergence est d'ordre p et de facteur C .

Si la méthode de la corde converge, alors cette convergence est linéaire. La méthode de la sécante admet une convergence d'ordre $p \in (1, 2)$. Si $f'(x^*) \neq 0$, alors la méthode de Newton-Raphson admet une convergence quadratique. La méthode de la fausse position admet également une convergence linéaire.

Exercice 10.1. Trouvez une racine de la fonction $f(x) = x^3 + 3x^2 - 1$ dans l'intervalle $[-2, 0]$ à l'aide de la méthode de la bisection. Faites 5 itérations.

Exercice 10.2. Que va-t-il se passer si on cherche une racine de la fonction $\tan(x)$ sur les intervalles suivants $I_1 = [1, 3]$ et $I_2 = [3, 4]$?

Exercice 10.3. Soit une fonction f continue et soit l'intervalle $I = [2, 7]$. Combien d'itérations de la méthode de la bisection sont nécessaires pour garantir une précision de 5×10^{-8} ?

Exercice 10.4. Trouvez une racine de la fonction $f(x) = x^3 - x^2 - 4x - 3$ dans l'intervalle $[2, 3]$ avec la méthode de la corde. Faites 5 itérations et prenez $x^{(0)} = \frac{5}{2}$.

Exercice 10.5. Trouvez une racine de la fonction $f(x) = 6x^3 - 23x^2 + 20x$ dans l'intervalle $[1, 2]$ avec la méthode de la sécante (faites 4 itérations).

Exercice 10.6. Déterminez une racine des fonctions suivantes dans les intervalles donnés via la méthode de la fausse position en 4 itérations :

1. $f(x) = \exp(x) - 2 - x$ et $I = [-2.4, -1.6]$;
2. $f(x) = \cos x + 1 - x$ et $I = [0.8, 1.6]$;
3. $f(x) = \ln x - 5 + x$ et $I = [3.2, 4]$;
4. $f(x) = x^2 - 10x + 23$ et $I = [6, 6.8]$.

Exercice 10.7. Trouvez une racine de la fonction $f(x) = x^3 - 9x + 2$ dans l'intervalle $I = [-4, -3]$ avec la méthode de Newton-Raphson. Faites 4 itérations et prenez $x^{(0)} = -\frac{7}{2}$.

Séance 11 — Résolution d'équations non-linéaires par itération de point fixe

Rappel théorique

Un raffinement itératif peut être vu comme une application répétée d'une fonction ϕ :

$$x^{(k+1)} = \phi(x^{(k)}).$$

Dans ce formalisme, trouver une solution à $f(x) = 0$ correspond à trouver un *point fixe* de la fonction ϕ , i.e. un point x tel que $\phi(x) = x$.

Théorème 11. Soit $\phi \in \mathcal{C}^1([a, b], \mathbb{R})$.

1. Si $\phi([a, b]) \subseteq [a, b]$, alors ϕ admet un point fixe dans l'intervalle $[a, b]$.
2. Si en plus $\phi' < 1$ sur $[a, b]$, alors pour tout choix de $x^{(0)} \in [a, b]$, la suite $(x^{(k)})_k$ des applications itérées de ϕ converge vers l'unique point fixe de ϕ sur $[a, b]$.
3. Si par contre $\phi' > 1$ sur $[a, b]$, alors pour tout choix de $x^{(0)}$ qui n'est pas un point fixe de ϕ , la suite $(x^{(k)})_k$ des applications itérées de ϕ ne converge pas.

Notons que par contraposée, le point 3 de ce théorème nous dit que pour avoir convergence, il faut nécessairement que $|\phi'(x^*)| \leq 1$.

Théorème 12 (Ostrowski). Soit $\phi \in \mathcal{C}^1([a, b], \mathbb{R})$ et soit $x^* \in [a, b]$, un point fixe de ϕ . Si $|\phi'(x^*)| < 1$, alors il existe $\delta > 0$ tel que pour tout $x^{(0)} \in (x^* \pm \delta)$, la limite suivante est vérifiée :

$$x^{(k)} \xrightarrow[k \rightarrow +\infty]{} x^*.$$

Si la dérivée de ϕ est trop grande en valeur absolue autour du point fixe, nous pouvons toujours changer de fonction ϕ . De manière générale, si nous voulons résoudre une équation de la forme $f(x) = 0$, pour toute fonction continue F telle que $F(0) = 0$, alors nous pouvons définir $\phi = \phi_F$ comme ceci :

$$\phi(x) = F(f(x)) + x.$$

De fait, si x^* satisfait $f(x^*) = 0$, alors :

$$\phi(x^*) = F(f(x^*)) + x^* = F(0) + x^* = x^*,$$

et donc x^* est un point fixe de ϕ . Attention cependant : si F s'annule en un point $z \neq 0$ et s'il existe un $\hat{x} \neq x^*$ tel que $f(\hat{x}) = z$, alors $\phi(\hat{x}) = \hat{x}$ (donc $\hat{x}$ est un point fixe de ϕ), même si $f(\hat{x}) = z \neq 0$.

Exercice 11.1. Trouvez un zéro de $f(x) = 2x^2 - 5x + 1$ en trouvant un point fixe. Utilisez d'abord 0, puis 1 comme valeur initiale. Faites 5 itérations.

Exercice 11.2. Déterminez si les fonctions suivantes admettent un point fixe sur l'intervalle donné :

- $\phi_1(x) = 1 - \frac{x^2}{4}$ sur $[0, 1]$;
- $\phi_2(x) = 2^{-x}$ sur $[0, 1]$;
- $\phi_3(x) = x^{-1}$ sur $[0.5, 5.2]$;

Exercice 11.3. Soit la fonction ϕ suivante :

$$\phi(x) = -\frac{1}{2}x^2 + 4x - 4.$$

Que pouvez-vous dire du comportement de l'itération de point fixe quand :

1. Résolvez $\phi(x) = x$ et montrez que 2 et 4 sont les seuls points fixes de ϕ .
2. Utilisez $x^{(0)} = 1.9$ pour l'itération de point fixe en 3 itérations.
3. Utilisez $x^{(0)} = 3.8$ pour l'itération de point fixe en 3 itérations.
4. Que peut-on en conclure ? Comment peut-on le vérifier ?

Exercice 11.4. Soit $f(x) = x^2 + x - 4$. Peut-on utiliser une itération de point fixe pour trouver les solutions de l'équation $f(x) = 0$? Pourquoi ?

Exercice 11.5. Trouver une racine de $f(x) = x - x^3$ en utilisant une itération de point fixe.